

Lecture 26: Machine Learning Continued

Heather Mattie, PhD

December 9, 2025



Recipe of the Day!

"The Best Chili Recipe"



Emre says that if finals have you stressed out to do something silly to make you laugh!

Announcements

- Lab this week on machine learning
- Homework #5 due Friday, December 12th
- Project files due Thursday, December 18th by 12pm (noon)
- There will be lectures held next week
 - Text mining
 - Next steps in data science
- No TF office hours next week – just Heather's
- No lab next week

Machine Learning

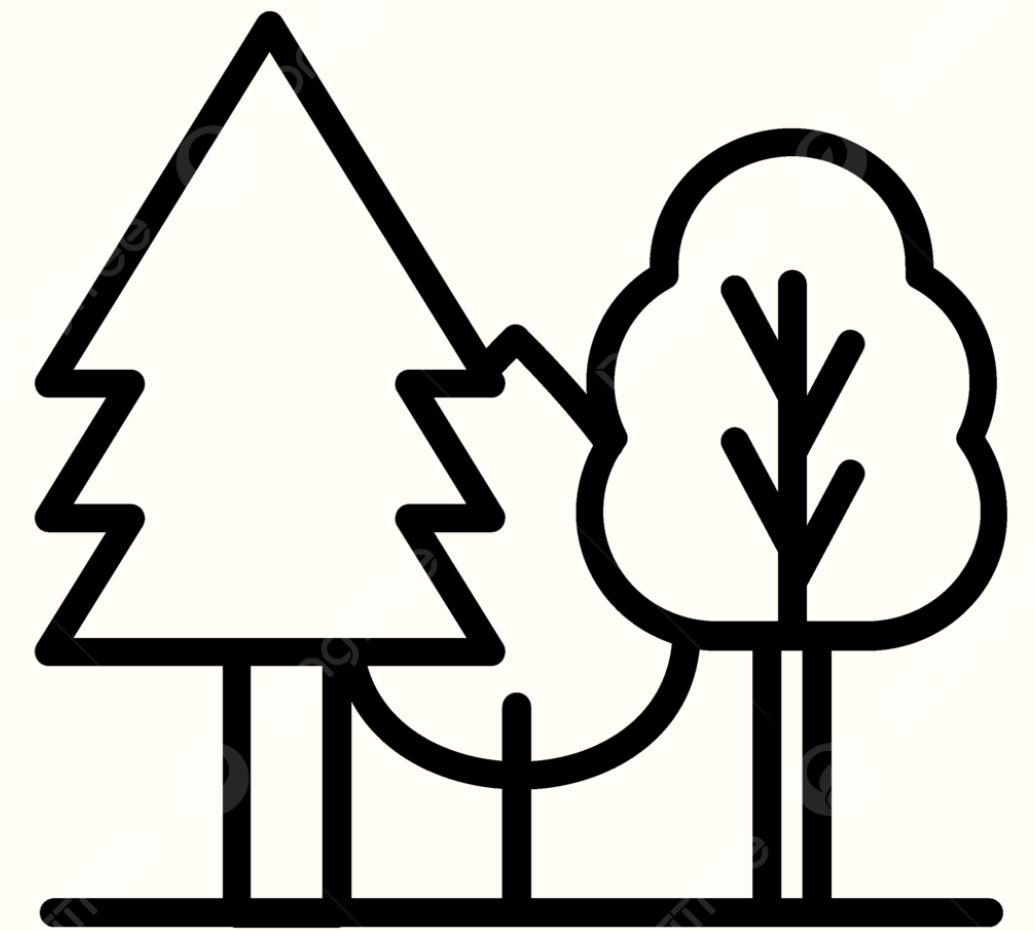
Random Forests

Heather Mattie, PhD

Random Forests

A **Random Forest (RF)** is an ensemble machine learning model that combines multiple decision trees to make more accurate and robust predictions. It is widely used for both classification and regression tasks.

- Overcomes the limitations of singular decision trees
- Ensemble method because it combines multiple decision trees to make predictions



Bagging

Bagging (bootstrap aggregation) is an ensemble technique used within Random Forests to improve predictive accuracy and reduce overfitting.

1. Build many decision trees from the training set
2. Given a new observation, calculate the prediction of the outcome for each tree
3. Aggregate the predictions; take the average for regression and the majority class for classification

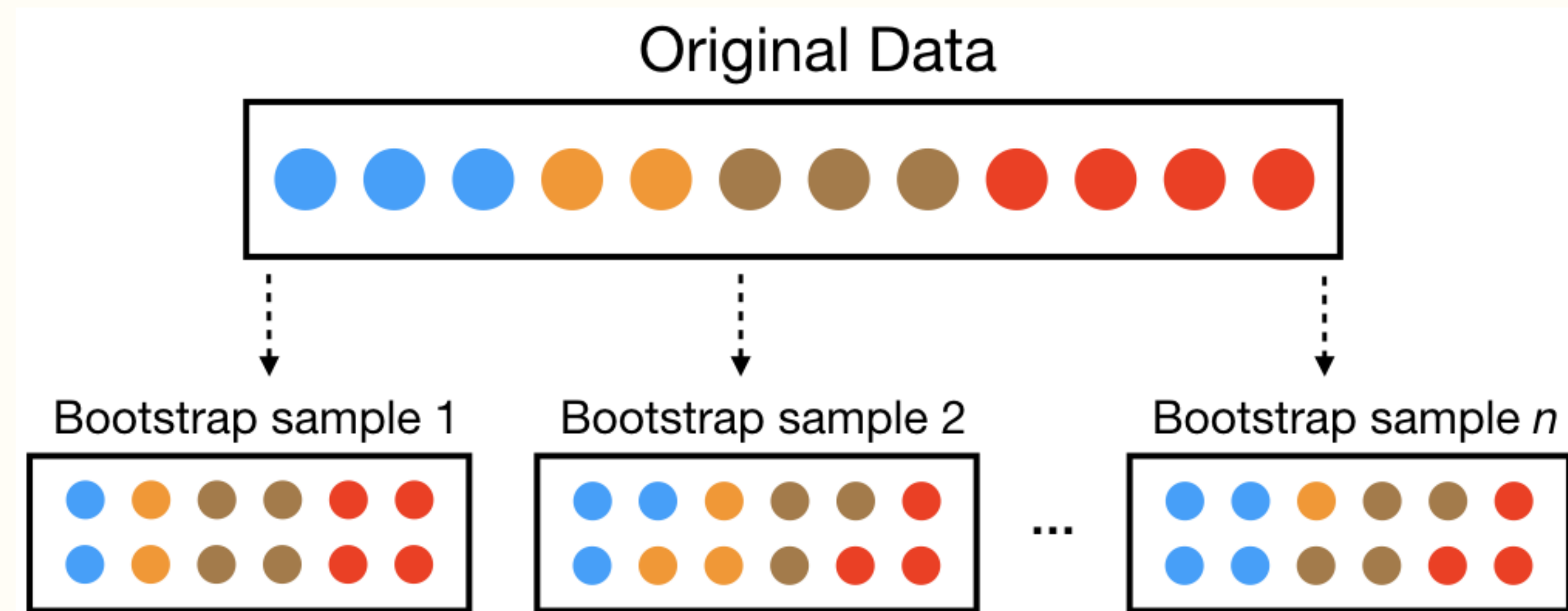
Bootstrap sampling

Bootstrap sampling creates multiple subsets of the training data by randomly sampling **with replacement** from the **original dataset**.

- Some data points may appear multiple times in a single bootstrap sample
- Some data points may be excluded entirely

Each of these distinct **bootstrap samples** is then used to train an **individual decision tree**.

Bootstrap sampling

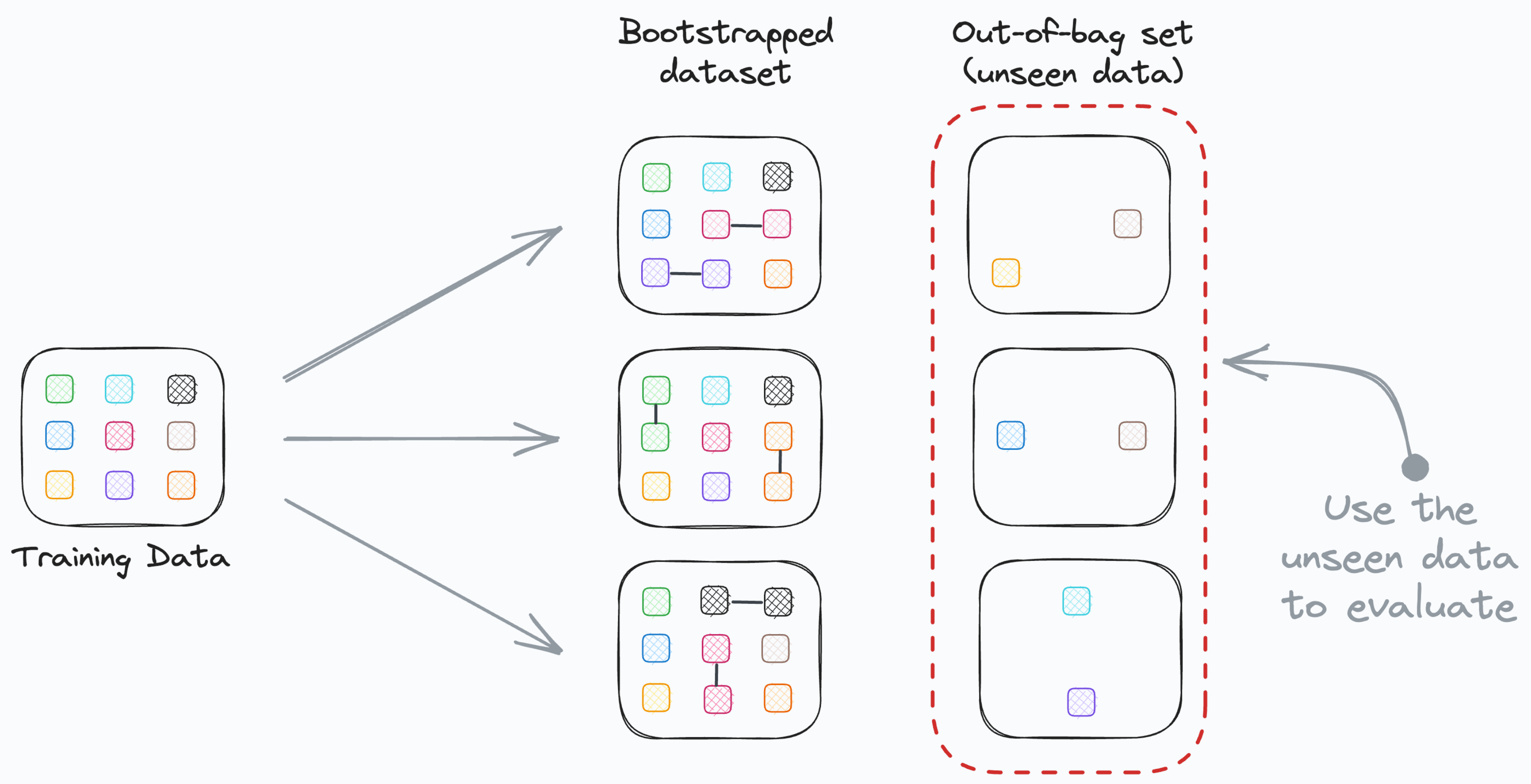


Out-of-bag (OOB)

Out-of-Bag (OOB) Samples: The instances that are *not* selected for a particular tree are called its out-of-bag (OOB) samples.

OOB Prediction: For each data point, the OOB samples are predicted by all the trees that were *not* trained on that data point. These individual OOB predictions are then aggregated (e.g., by averaging for regression or majority voting for classification) to form a final OOB prediction for that instance.

Out-of-bag (OOB)



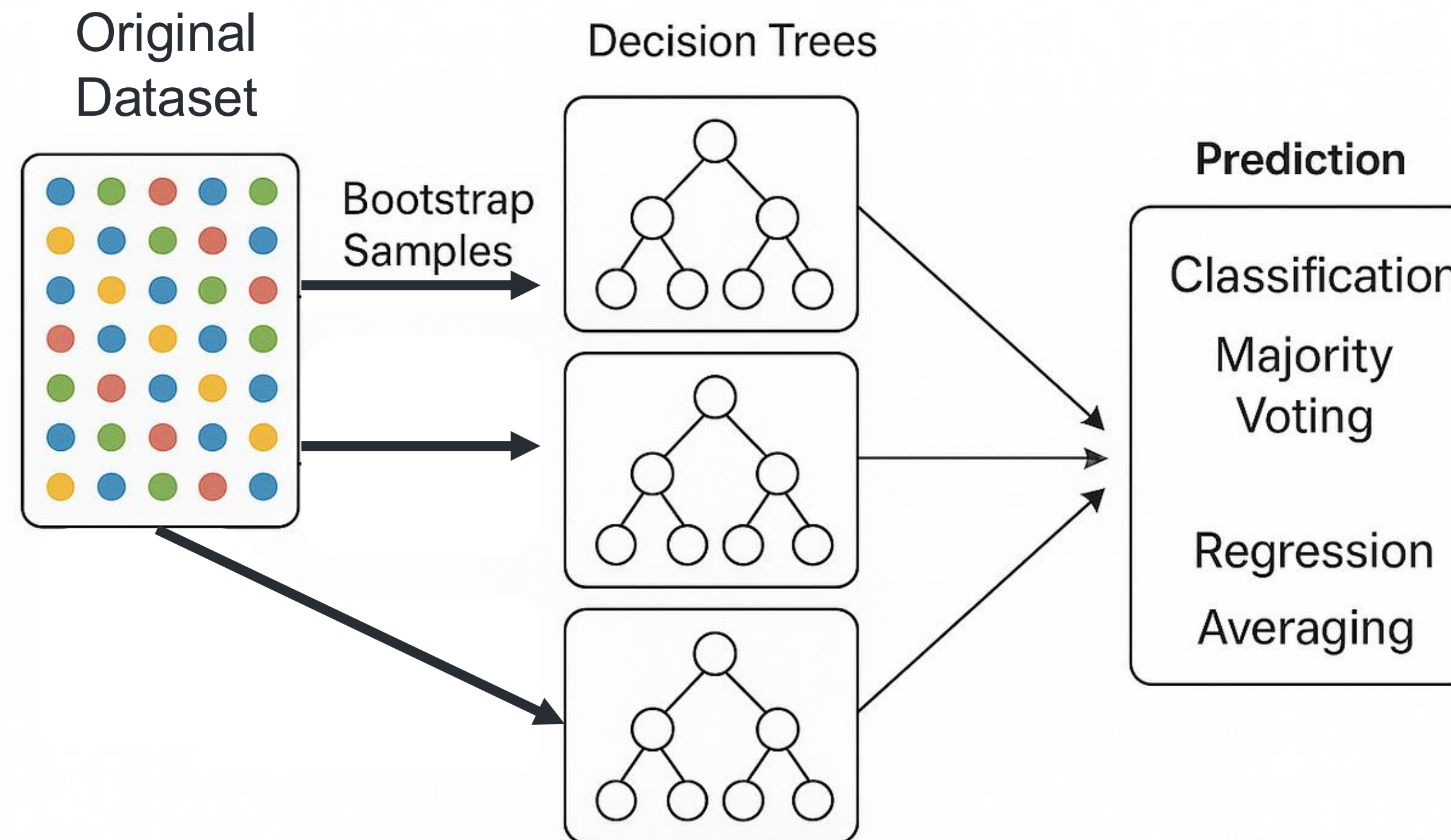
Aggregation

After each decision tree is trained on its respective bootstrap sample, the predictions from all the individual trees are combined (**aggregated**) to form a final prediction.

- For classification tasks: majority vote among the trees
- For regression tasks: the average of the predictions from all trees

Aggregation

Random Forest Regression and Classification



Feature bagging

Random Forests build upon bagging by adding an extra layer of randomness during the tree construction process. In addition to bootstrap sampling, Random Forests also randomly select a subset of features at each split point within each individual decision tree.

This “**feature bagging**” further **decorrelates** the trees, leading to even greater variance reduction and often improved predictive performance compared to traditional bagging alone.

Problems addressed

- **Variance reduction:** By training multiple trees on different subsets of the data, bagging helps to reduce the variance of the model. Each tree learns slightly different patterns due to the variations in the bootstrap samples, leading to a more robust and less sensitive overall model.
- **Overfitting mitigation:** The diversity introduced by bootstrap sampling helps to prevent individual trees from overfitting to the training data. The aggregation of predictions from diverse trees further reduces the risk of overfitting by smoothing out individual tree errors.

Example: Boston Housing

The **Boston Housing** dataset contains information collected by the U.S Census Service concerning housing in the area of Boston, MA in the 1970s. There are 14 variables total. Below are details about a few of them.

- **medv**: median value of owner-occupied homes in \$1000's
- **lstat**: % of individuals with lower socioeconomic status
- **rm**: average number of rooms per dwelling
- **dis**: weighted distances to five Boston employment centres
- **nox**: nitric oxides concentration (parts per 10 million)
- **crim**: per capita crime rate by town

The `randomForest` Function

The `randomForest` function from the `randomForest` package trains Random Forest models.

- **`mtry`**: number of predictors (p) sampled for splitting each node, default is $p/3$ for regression and $\sqrt{p}$ for classification
- **`ntree`**: number of trees grown, default is 500
- **`nodesize`**: minimum size of terminal nodes (how many observations in each terminal leaf)

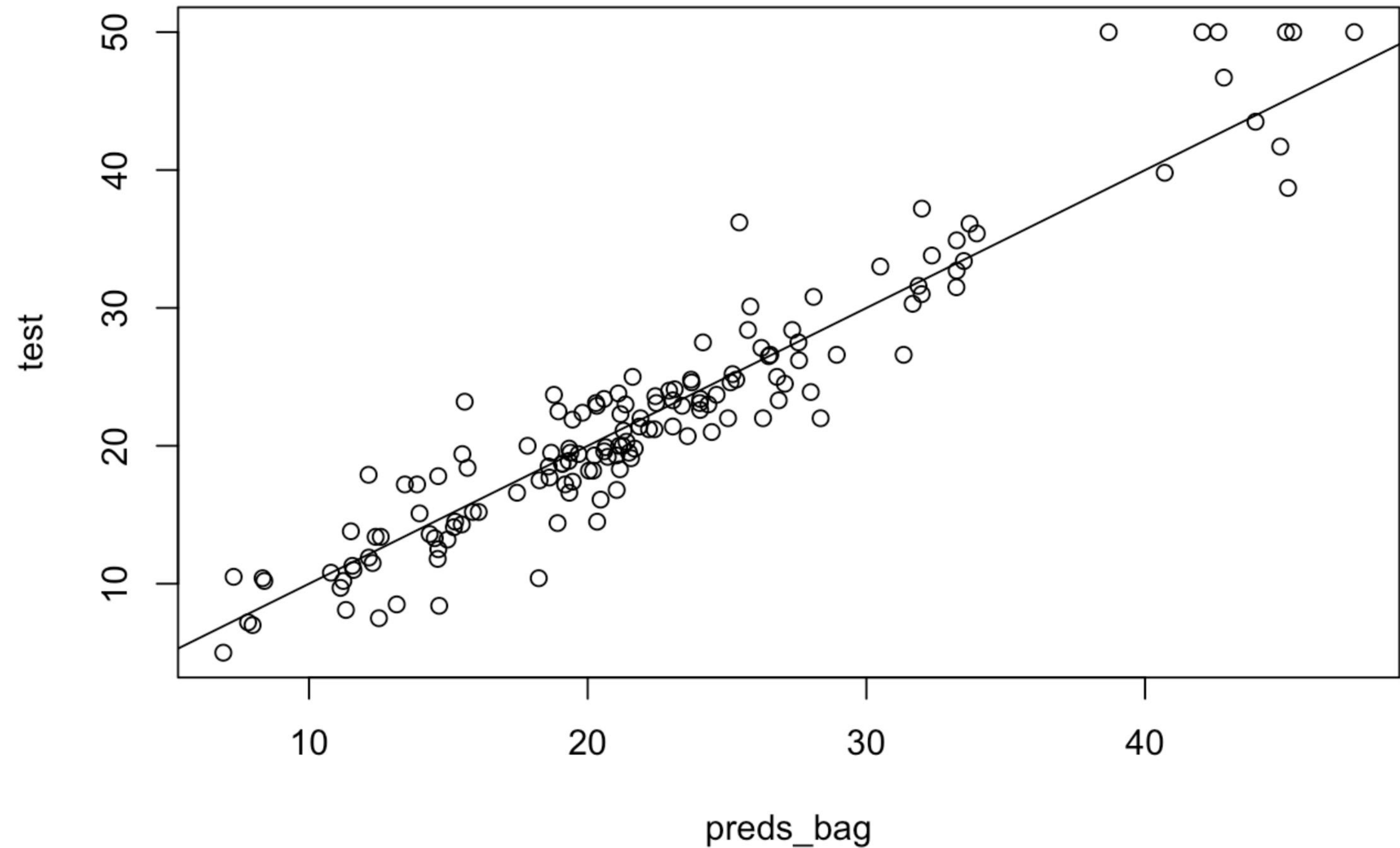
```
library(randomForest)
library(MASS)
fit_bag <- randomForest(medv ~ ., data = train_set, mtry = 13)
fit_bag
```

```
##
## Call:
## randomForest(formula = medv ~ ., data = train_set, mtry = 13)
##               Type of random forest: regression
##               Number of trees: 500
## No. of variables tried at each split: 13
##
##               Mean of squared residuals: 13.68988
##               % Var explained: 83.29
```

Making predictions

- The **predict** function is used to make predictions
- The **plot** function can be used to display the predictions and ground truth values

```
preds_bag = predict(fit_bag, newdata = test_set)
plot(preds_bag, test)
abline(0,1)
```



Evaluating performance

- The MSE/RMSE are much lower compared to the single decision trees.
- This model leads to test predictions that are within approximately \$3,021 of the true median home value.

```
mean((preds_bag - test)^2)
```

```
## [1] 9.12901
```

```
sqrt(mean((preds_bag - test)^2))
```

```
## [1] 3.021425
```

Changing `mtry`

Reducing the size of the subset of predictors chosen for each split in a tree can help prevent overfitting.

Here, it does not improve performance on the test set.

```
fit_rf <- randomForest(medv ~ ., data = train_set, mtry = 6)
preds_rf = predict(fit_rf, newdata = test_set)
mean((preds_rf - test)^2)
```

```
## [1] 10.00925
```

```
sqrt(mean((preds_rf - test)^2))
```

```
## [1] 3.16374
```

Finding the optimal **mtry** value

The train function can be used similarly to when cross-validation was used for other models.

- Provide a list of values to try to the **tuneGrid** argument

```
# Define the tuning grid for mtry (numbers 2-13)
tune_grid <- expand.grid(mtry = seq(2, 13))

# Set up training control (e.g., repeated cross-validation)
train_control <- trainControl(method = "cv",
                              number = 10)

# Train the random forest model with tuning
rf_tuned_model <- train(medv ~ .,
                        data = train_set,
                        method = "rf",
                        tuneGrid = tune_grid,
                        trControl = train_control)

# View the results and optimal mtry
print(rf_tuned_model)
```

Finding the optimal `mtry` value

The optimal value for `mtry` turned out to be 6 – the second Random Forest we trained.

```
## Random Forest
##
## 356 samples
## 13 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 320, 321, 321, 320, 320, 321, ...
## Resampling results across tuning parameters:
##
##   mtry  RMSE      Rsquared  MAE
##   2     3.820984  0.8346431  2.519514
##   3     3.618008  0.8447581  2.388614
##   4     3.545530  0.8455778  2.353909
##   5     3.508104  0.8463937  2.326586
##   6     3.501270  0.8456823  2.325723
##   7     3.517672  0.8423850  2.332987
##   8     3.539399  0.8405534  2.355711
##   9     3.573090  0.8372022  2.355729
##  10     3.607730  0.8341349  2.367331
##  11     3.640190  0.8294652  2.387899
##  12     3.716452  0.8239906  2.400136
##  13     3.690969  0.8263174  2.405034
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 6.
```

Comparing performance

Model	# Terminal Leaves	MSE	RMSE
Full tree with <code>tree</code> function	9	19.10	4.37
Pruned tree with <code>tree</code> function	5	24.85	4.98
Full tree with <code>rpart</code> function	8	19.44	4.41
Pruned tree with <code>rpart</code> function	5	24.85	4.98
Random Forest with <code>mtry</code> = 13	-	9.13	3.02
Random Forest with <code>mtry</code> = 6	-	10.01	3.16



Variable importance

Variable importance quantifies how predictive each feature was during training.

Two common methods:

- Gini importance looks at how much a variable improves node purity across all splits in the forest.
- Permutation importance assesses the drop in accuracy when a variable's values are randomly permuted.
- A larger value indicates more predictive.

```
library(knitr)
variable_importance <- importance(fit_rf)
tmp <- tibble(feature = rownames(variable_importance),
              Gini = variable_importance[,1]) %>%
              arrange(desc(Gini))

kable(tmp[1:10,])
```

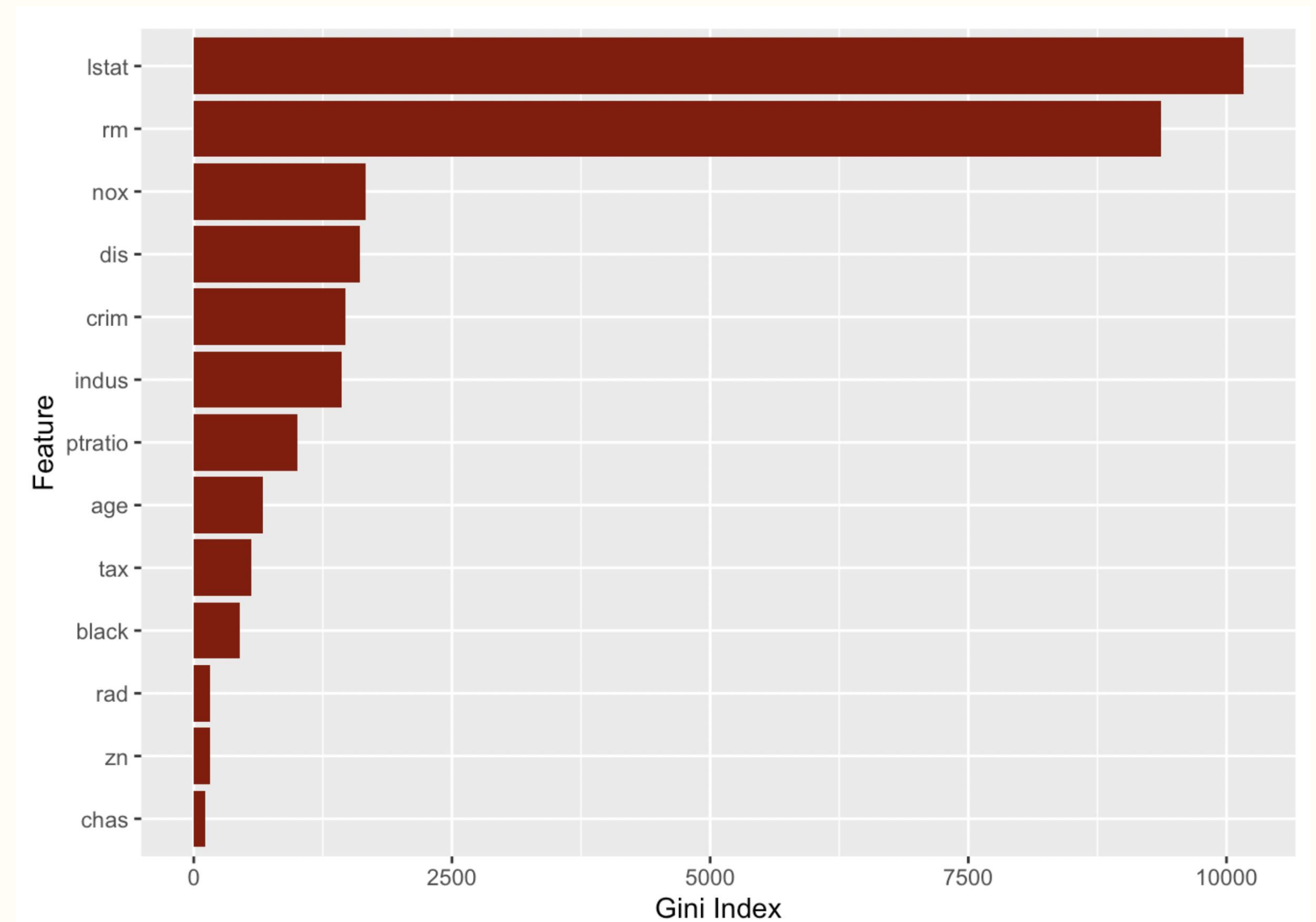
feature	Gini
lstat	10163.4095
rm	9364.2909
nox	1667.2821
dis	1604.2735
crim	1464.4502
indus	1429.4463
ptratio	1003.1227
age	672.2566
tax	557.5735
black	446.2853



Variable importance

The two most predictive features are:

- **lstat**: % of individuals with lower socioeconomic status
- **rm**: average number of rooms per dwelling



Classification example

Using the **Pima Indians diabetes dataset** from the **mlbench** package we will predict diabetes status, positive or negative for diabetes, using all other predictors available in the dataset.

- **pregnant**: Number of times pregnant
- **glucose**: Plasma glucose concentration (glucose tolerance test)
- **pressure**: Diastolic blood pressure (mm Hg)
- **triceps**: Triceps skin fold thickness (mm)
- **insulin**: 2-Hour serum insulin (mu U/ml)
- **mass**: Body mass index (BMI)
- **pedigree**: Diabetes pedigree function (a genetic risk score)
- **age**: Age (years)
- **diabetes**: indicator of diabetes

Example: Pima Indians

```
library(mlbench)
data("PimaIndiansDiabetes2")

PimaIndiansDiabetes2 <- PimaIndiansDiabetes2

head(PimaIndiansDiabetes2)
```

##	pregnant	glucose	pressure	triceps	insulin	mass	pedigree	age	diabetes
## 1	6	148	72	35	NA	33.6	0.627	50	pos
## 2	1	85	66	29	NA	26.6	0.351	31	neg
## 3	8	183	64	NA	NA	23.3	0.672	32	pos
## 4	1	89	66	23	94	28.1	0.167	21	neg
## 5	0	137	40	35	168	43.1	2.288	33	pos
## 6	5	116	74	NA	NA	25.6	0.201	30	neg

Missingness

There is missingness in this dataset that needs to be handled.
Here, we will use median imputation for all missing values.

```
colSums(is.na(PimaIndiansDiabetes2))
```

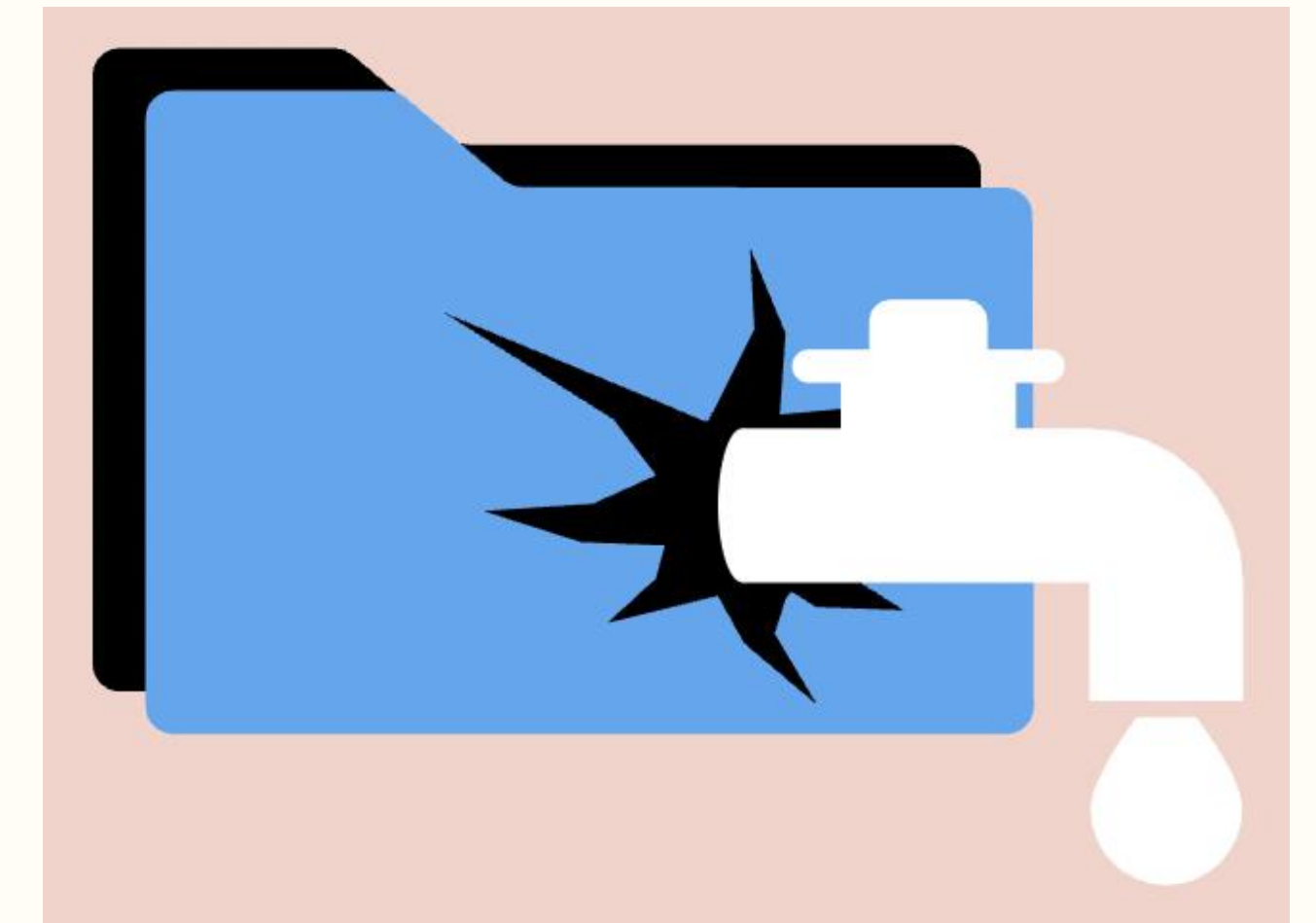
```
## pregnant  glucose  pressure  triceps  insulin    mass  pedigree    age
##          0         5        35      227     374     11         0      0
## diabetes
##          0
```

Data leakage

Imputation, like the pre-processing used for KNN, must be done using only the training set, to avoid data leakage.

Data leakage refers to the use of information about the test set that is used when training the model.

- This can lead to overly optimistic model performance because the model has “peeked” at the test set data during training
- Imputation performed on the whole dataset before splitting into training and test sets will cause data leakage



Imputation steps

1. Split data into training and test sets as usual.
2. Subset the training and test sets into data frames with just the predictors and just the outcome. Only the predictors will be used for imputation. Here it is assumed that there is no missingness for the outcome variable.
3. Fit imputation model using ONLY the training data.
4. Apply the imputation model to the training and test sets.
5. Reattach the predictors and outcome.

Step 1

- Split data into training and test sets as usual

```
library(splitstackshape)
set.seed(1)

x      <- stratified(PimaIndiansDiabetes2, "diabetes", 0.7, keep.rownames = TRUE)
train_set <- x %>% dplyr::select(-rn)
train_index <- as.numeric(x$rn)
test_set  <- PimaIndiansDiabetes2[-train_index,]
test_set_y <- test_set$diabetes
dim(train_set)
```

```
## [1] 538  9
```

```
dim(test_set)
```

```
## [1] 230  9
```

Step 2

- Subset the training and test sets into data frames with just the predictors and just the outcome. Only the predictors will be used for imputation. Here it is assumed that there is no missingness for the outcome variable.

```
# Separate predictors from outcome for imputation  
train_x <- train_set %>% dplyr::select(-diabetes)  
train_y <- train_set$diabetes  
  
test_x <- test_set %>% dplyr::select(-diabetes)  
test_y <- test_set$diabetes
```

Step 3

- Fit imputation model using ONLY the training data.
 - Median imputation is used here with the **preProcess** function from the **caret** package, but any method can be used.

```
# Fit imputation model on training data predictors only  
imputation_model <- preProcess(train_x, method = "medianImpute")
```

Step 4

- Apply the imputation model to the training and test sets

```
# Apply same imputation model on the training and test sets  
train_x_imp <- predict(imputation_model, train_x)  
test_x_imp  <- predict(imputation_model, test_x)
```

Step 5

- Reattach the predictors and outcome.

```
# Re-combine with outcome
pima_imputed_train <- train_x_imp %>%
  mutate(diabetes = train_y)

pima_imputed_test <- test_x_imp %>%
  mutate(diabetes = test_y,
         diabetes_num = as.factor(ifelse(diabetes == "neg", 0, 1)))
```

Define cross-validation function

Cross-validation will be used to find the best value for **mtry**, the number of predictors considered at each split in a single decision tree.

```
set.seed(1)
tune_grid <- expand.grid(mtry = seq(2, 8))

train_control <- trainControl(method = "cv",
                              number = 10,
                              classProbs = TRUE,
                              summaryFunction = twoClassSummary)
```

Model training

```
rf_tuned_model <- train(diabetes ~ .,  
                        data      = pima_imputed_train,  
                        method    = "rf",  
                        tuneGrid  = tune_grid,  
                        trControl = train_control,  
                        metric    = "ROC")
```

rf_tuned_model

The final value of **mtry** used is 5.

```
## Random Forest  
##  
## 538 samples  
## 8 predictor  
## 2 classes: 'neg', 'pos'  
##  
## No pre-processing  
## Resampling: Cross-Validated (10 fold)  
## Summary of sample sizes: 484, 485, 484, 485, 484, 484, ...  
## Resampling results across tuning parameters:  
##  
##   mtry  ROC      Sens      Spec  
##   2    0.8612907 0.8800000 0.6494152  
##   3    0.8606182 0.8685714 0.6652047  
##   4    0.8619298 0.8628571 0.6757310  
##   5    0.8630535 0.8600000 0.6809942  
##   6    0.8589348 0.8485714 0.6707602  
##   7    0.8572139 0.8428571 0.6760234  
##   8    0.8575940 0.8457143 0.6543860  
##  
## ROC was used to select the optimal model using the largest value.  
## The final value used for the model was mtry = 5.
```

Model performance

```
p_hat_rf = predict(rf_tuned_model, newdata = pima_imputed_test, type = "prob")[,2]
y_hat_rf = factor(ifelse(p_hat_rf > 0.5, 1, 0))

confusionMatrix(data = as.factor(y_hat_rf),
                 reference = pima_imputed_test$diabetes_num,
                 positive = "1")
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##              0 117  32
##              1  33  48
##
##              Accuracy : 0.7174
##              95% CI : (0.6545, 0.7746)
##              No Information Rate : 0.6522
##              P-Value [Acc > NIR] : 0.02107
##
##              Kappa : 0.3789
##
##              Mcnemar's Test P-Value : 1.00000
##
##              Sensitivity : 0.6000
##              Specificity : 0.7800
```

Decision tree comparison

A single decision tree can be trained to compare performance. The same cross-validation specifications are used here.

- **method = "rpart"** designates an **rpart** tree as the model
- **tuneLength = 10** performs cross-validation for the complexity parameter. 10 indicates the number of parameters to try

```
set.seed(1)

# Fit a regression tree using all of the available predictors
rpart_tuned_model <- train(diabetes ~ .,
                           data      = pima_imputed_train,
                           method    = "rpart",
                           trControl = train_control,
                           tuneLength = 10, # caret will try different complexity parameters
                           metric     = "ROC")

rpart_tuned_model
```

Decision tree comparison

A single decision tree can be trained to compare performance. The same cross-validation specifications are used here.

- A complexity parameter of 0 was used for the final tree.

```
## CART
##
## 538 samples
## 8 predictor
## 2 classes: 'neg', 'pos'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 484, 485, 484, 485, 484, 484, ...
## Resampling results across tuning parameters:
##
##   cp          ROC        Sens       Spec
## 0.00000000 0.7787636 0.8371429 0.6119883
## 0.03546099 0.7424896 0.8657143 0.5210526
## 0.07092199 0.6698663 0.9514286 0.3883041
## 0.10638298 0.6698663 0.9514286 0.3883041
## 0.14184397 0.6698663 0.9514286 0.3883041
## 0.17730496 0.6698663 0.9514286 0.3883041
## 0.21276596 0.6698663 0.9514286 0.3883041
## 0.24822695 0.6698663 0.9514286 0.3883041
## 0.28368794 0.6698663 0.9514286 0.3883041
## 0.31914894 0.5599708 0.9600000 0.1599415
##
## ROC was used to select the optimal model using the largest value.
## The final value used for the model was cp = 0.
```

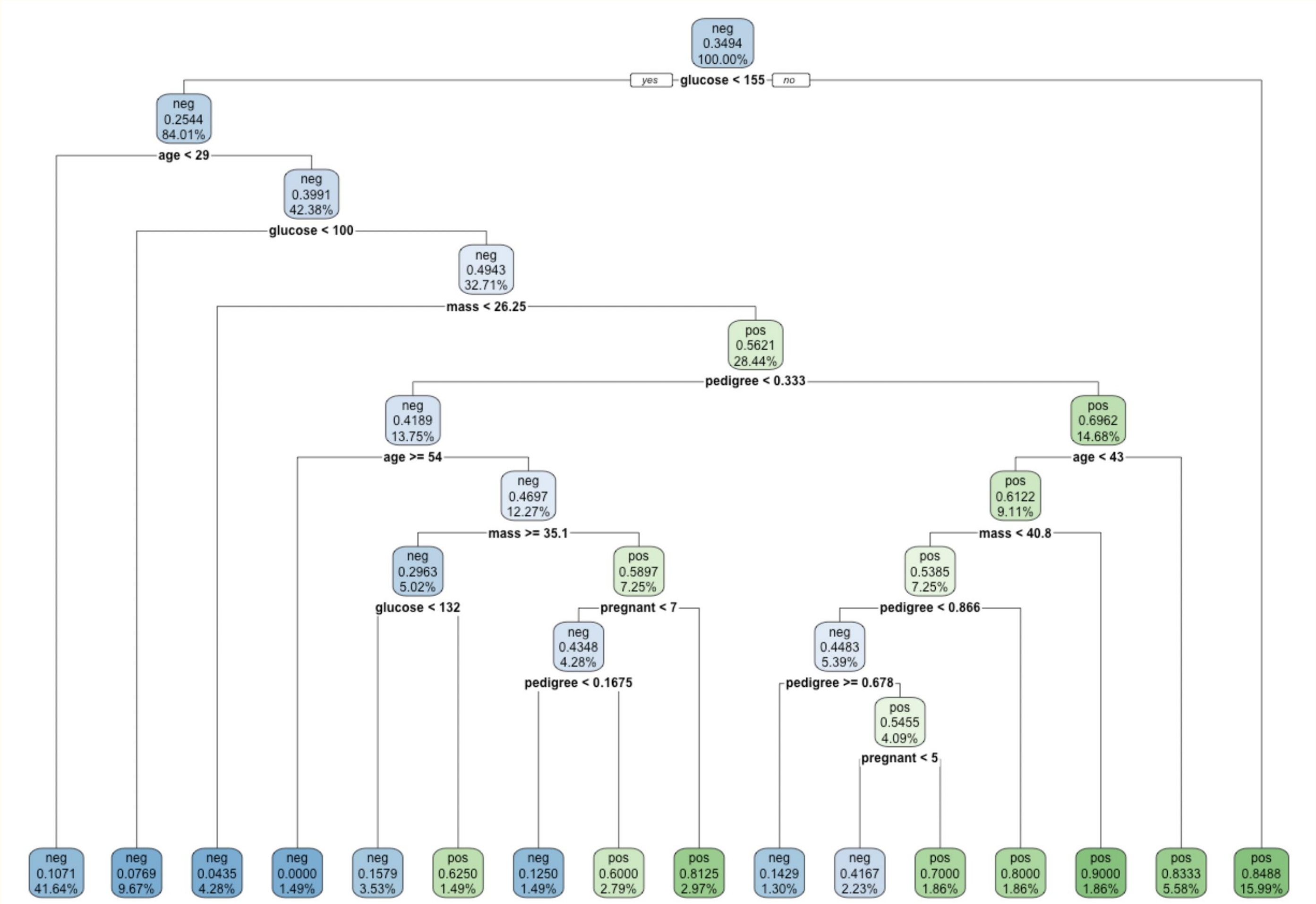
Final decision tree

The final tree is only one part of the output produced by the train function.

- **finalModel** is used to access the final tree

```
final_tree <- rpart_tuned_model$finalModel  
rpart.plot(final_tree, digits = 4)
```

Final decision tree



Final decision tree

The `final.rpart` function can be used to display the logic of the tree.

```
kable(rpart.rules(final_tree))
```

.outcome																													
92	0.00	when	glucose	is	100	to	155	&	age	>=	54	&	mass	>=	26	&	pedigree	<	0.33										
22	0.04	when	glucose	is	100	to	155	&	age	>=	29	&	mass	<	26														
10	0.08	when	glucose	<	100				&	age	>=	29																	
4	0.11	when	glucose	<	155				&	age	<	29																	
748	0.12	when	glucose	is	100	to	155	&	age	is	29	to	54	&	mass	is	26	to	35	&	pedigree	<	0.17			&	pregnant	<	7
752	0.14	when	glucose	is	100	to	155	&	age	is	29	to	43	&	mass	is	26	to	41	&	pedigree	is	0.68	to	0.87				
372	0.16	when	glucose	is	100	to	132	&	age	is	29	to	54	&	mass	>=	35	&	pedigree	<	0.33								
1506	0.42	when	glucose	is	100	to	155	&	age	is	29	to	43	&	mass	is	26	to	41	&	pedigree	is	0.33	to	0.68	&	pregnant	<	5
749	0.60	when	glucose	is	100	to	155	&	age	is	29	to	54	&	mass	is	26	to	35	&	pedigree	is	0.17	to	0.33	&	pregnant	<	7
373	0.62	when	glucose	is	132	to	155	&	age	is	29	to	54	&	mass	>=	35	&	pedigree	<	0.33								
1507	0.70	when	glucose	is	100	to	155	&	age	is	29	to	43	&	mass	is	26	to	41	&	pedigree	is	0.33	to	0.68	&	pregnant	>=	5
377	0.80	when	glucose	is	100	to	155	&	age	is	29	to	43	&	mass	is	26	to	41	&	pedigree	>=	0.87						
375	0.81	when	glucose	is	100	to	155	&	age	is	29	to	54	&	mass	is	26	to	35	&	pedigree	<	0.33			&	pregnant	>=	7
95	0.83	when	glucose	is	100	to	155	&	age	>=	43	&	mass	>=	26	&	pedigree	>=	0.33										
3	0.85	when	glucose	>=	155																								
189	0.90	when	glucose	is	100	to	155	&	age	is	29	to	43	&	mass	>=	41	&	pedigree	>=	0.33								



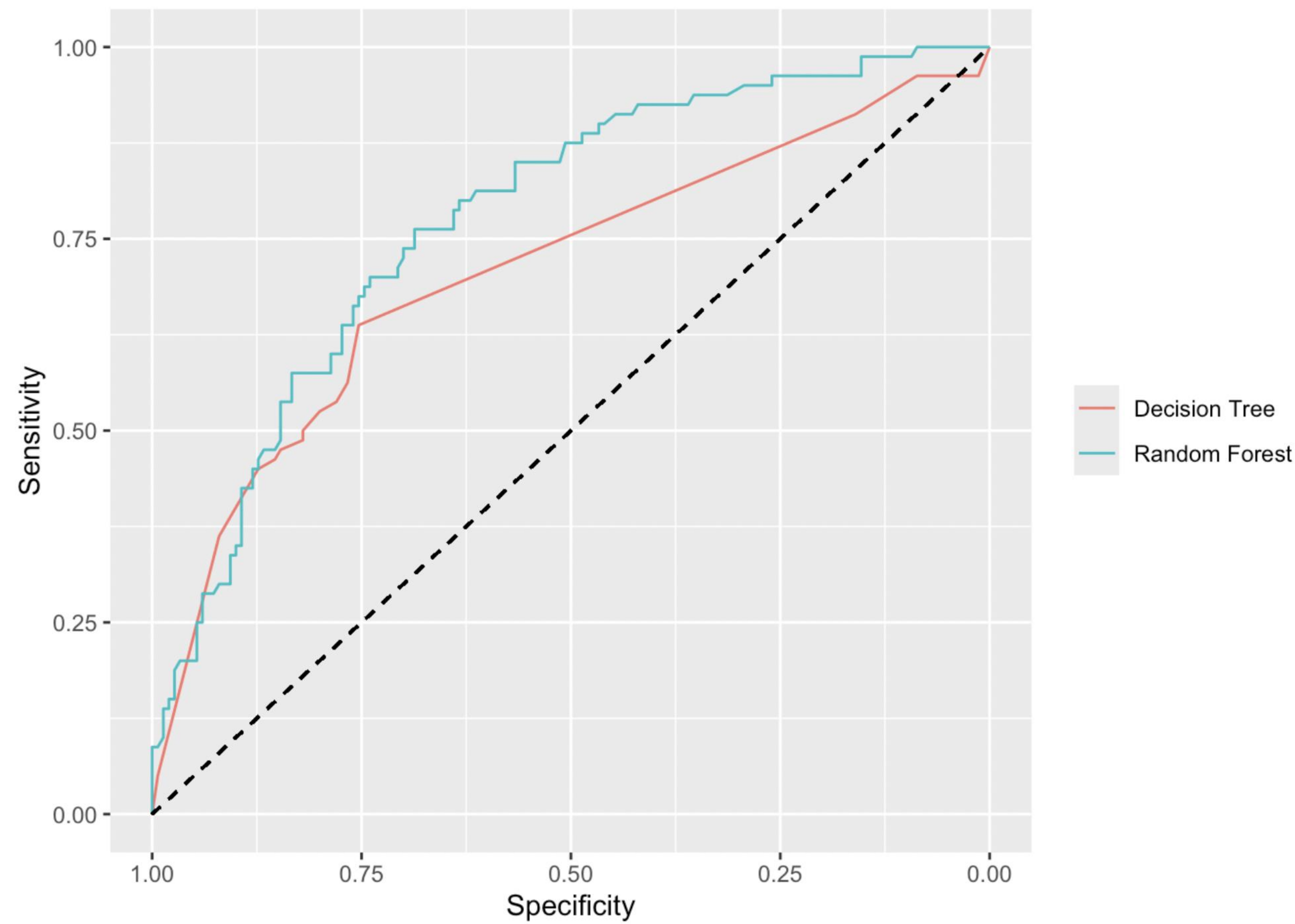
Decision tree performance

```
# Use tree for prediction
p_hat_rpart= predict(rpart_tuned_model, newdata = pima_imputed_test, type = "prob")[,2]
y_hat_rpart = factor(ifelse(p_hat_rpart > 0.5, 1, 0))

confusionMatrix(data = as.factor(y_hat_rpart),
                 reference = pima_imputed_test$diabetes_num,
                 positive = "1")
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##           0 120  38
##           1  30  42
##
##              Accuracy : 0.7043
##              95% CI : (0.6408, 0.7625)
##      No Information Rate : 0.6522
##      P-Value [Acc > NIR] : 0.05433
##
##              Kappa : 0.3328
##
##      McNemar's Test P-Value : 0.39595
##
##              Sensitivity : 0.5250
##              Specificity : 0.8000
```

Performance comparison



Model	Overall Accuracy	Sensitivity	Specificity
Random Forest	0.72	0.60	0.78
Decision tree	0.70	0.53	0.80

```
auc(roc_tree)
```

```
## Area under the curve: 0.7078
```

```
auc(roc_rf)
```

```
## Area under the curve: 0.7763
```

Random Forest summary

Pros

- High accuracy
- Handles complex relationships automatically
- Resistant to noise and outliers

Cons

- Harder to interpret than one decision tree
- More computationally intensive